

PID-Temperaturregelung mit ESP32

Oct 20, 2026 · @Reinhard Wermeling

Überblick

Ein ESP32 hält eine Temperatur mit einem PID-Regler auf Sollwert: Er misst mit einem DS18B20, berechnet jede Sekunde einen Stellwert von 0–100 % und taktet damit eine Heizung über ein Solid-State-Relais (SSR).

Das Projekt besteht aus zwei Teilen:

Teil	Datei	Zweck
Arduino-Sketch	`PID_Temperatur.ino`	Läuft auf dem ESP32, misst, regelt, schaltet die Heizung
Browser-Simulator	PID-Temperatur Simulator (Artifact)	Lehrmodell mit derselben Regellogik, zum Ausprobieren ohne Hardware

Funktionsumfang des Sketches:

- PID-Regler mit Anti-Windup, D-Anteil auf den Istwert und stoßfreier Hand/Automatik-Umschaltung
- Nicht-blockierende Temperaturmessung (DS18B20, 12 bit, 0,0625 K Auflösung)
- Zeit-Proportional-Ansteuerung des SSR im 2-s-Fenster
- Sicherheitsabschaltung bei Übertemperatur (95 °C) und Sensorfehler, verriegelt bis zur Quittierung
- Bedienung und Parametrierung über den Seriellen Monitor, Parameter dauerhaft im Flash
- CSV-Ausgabe für den Seriellen Plotter der Arduino IDE

Ausgangspunkt war ein eigener PID-Baustein (`struct PID`). Er wurde übernommen, an einigen Stellen korrigiert und zu einem Gesamtprogramm ergänzt (Abschnitt *Der PID-Baustein*).

Hardware & Verdrahtung

Die Grundschaltung braucht nur drei Teile am ESP32: den Temperatursensor, ein SSR mit DC-Steuereingang und die Onboard-LED als Anzeige.

Bauteil	Anschluss	ESP32-Pin	Hinweis
DS18B20	VDD	3,3 V	Wasserdichte Ausführung für Flüssigkeiten
DS18B20	GND	GND	
DS18B20	DATA	GPIO 4	4,7 kΩ Pull-up nach 3,3 V
SSR	Steuer +	GPIO 26	SSR mit Eingang 3–32 V DC
SSR	Steuer –	GND	
LED	–	GPIO 2	Onboard, leuchtet bei Heizung ein, blinkt schnell bei Fehler

Die Pins sind im Sketch als Konstanten `PIN_ONEWIRE`, `PIN_HEIZUNG` und `PIN_LED` definiert und lassen sich dort ändern.

SSR-Auswahl. Für ohmsche Heizungen am Netz eignet sich ein SSR mit Nullspannungsschaltung (Zero-Cross), Lastnennstrom mindestens das 2-Fache des Heizstroms. Ab etwa 3 A Laststrom braucht das SSR einen Kühlkörper.

Sicherheit bei Netzspannung. Die Lastseite führt 230 V. Aufbau nur durch eine Elektrofachkraft oder unter ihrer Aufsicht, mit Gehäuse, Schutzleiter und Sicherung. Die Software-Abschaltung bei 95 °C ersetzt keinen Hardware-Schutz: Ein SSR fällt typischerweise *leitend* aus. Deshalb gehört ein unabhängiger Übertemperaturschutz (Thermosicherung oder Schutztemperaturbegrenzer) in den Lastkreis.

Software-Aufbau

Der Sketch läuft in der Arduino IDE mit dem ESP32-Core und braucht zwei Bibliotheken aus dem Bibliotheksverwalter.

Bibliothek	Autor	Zweck
OneWire	Paul Stoffregen	Busprotokoll für den DS18B20
DallasTemperature	Miles Burton	Temperatur lesen, Auflösung, asynchrone Messung
Preferences	im ESP32-Core	Parameter im Flash (NVS) speichern

Konfigurationskonstanten

Konstante	Wert	Bedeutung
`ABTASTZEIT_MS`	1000	Reglertakt; der DS18B20 braucht bei 12 bit 750 ms pro Messung
`PWM_FENSTER_MS`	2000	Zeitfenster der SSR-Taktung
`MIN_SCHALT_MS`	50	Kürzeste Ein- oder Auszeit, schont SSR und Netz
`T_MAX`	95,0 °C	Übertemperatur-Abschaltung
`SOLL_MIN` / `SOLL_MAX`	0 / 90 °C	Erlaubter Sollwertbereich

Programmablauf

`setup()` schaltet zuerst den Heizungsausgang auf LOW, damit die Heizung beim Booten sicher aus ist. Danach startet es Serial (115200 Baud), initialisiert den Sensor im nicht-blockierenden Modus, lädt die Parameter aus dem Flash und gibt den Status aus.

`loop()` arbeitet ohne `delay()` und ruft pro Durchlauf vier Funktionen auf:

1. `seriellLesen()` sammelt Zeichen und führt bei Enter einen Befehl aus.
2. `abtastung()` läuft genau alle 1000 ms: Messwert abholen, neue Messung starten, Stellwert berechnen, CSV-Zeile ausgeben.
3. `ssrAnsteuern()` setzt den Heizungsausgang laufend nach Stellwert und Position im 2-s-Fenster.
4. `ledAnzeigen()` spiegelt den Heizungsausgang oder blinkt im Fehlerfall.

Der Takt wird mit `letzteAbtastung += ABTASTZEIT_MS` fortgeschrieben statt auf `millis()` gesetzt. So bleibt das Raster auch bei kleinen Verzögerungen exakt, und der Regler sieht ein gleichmäßiges dt.

Der PID-Baustein

Der Regler berechnet aus Soll- und Istwert jeden Takt einen Stellwert, begrenzt auf 0–100 %. Er ist die Summe aus drei Anteilen:

$$u(t) = K_p \cdot e(t) + K_i \int e(t)\,dt - K_d \cdot \frac{dx(t)}{dt}, \qquad e = w - x$$

Dabei ist w der Sollwert, x der Istwert und e die Regelabweichung.

Anteil	Formel im Code	Wirkung
P	`kp * fehler`	Reagiert sofort auf die Abweichung; allein bleibt eine Restabweichung
I	`iAnteil += ki * fehler * dt`	Liefert die Halteleistung und beseitigt die Restabweichung
D	`-kd * (istwert - prevInput) / dt`	Bremst bei schnellem Anstieg und dämpft das Überschwingen

Korrekturen gegenüber dem Original

Stelle	Original	Neu	Grund
Ausreißer-Grenze	`dt > 1.0f` fest	`dt > dtMax` (5 s)	Bei 1 s Reglertakt lag dt oft knapp über 1,0, der Regler lieferte dann ständig 0
I-Speicher	`integral` = Σ Fehler·dt, `ki` erst bei der Ausgabe	`iAnteil` = Σ ki·Fehler·dt	Eine Änderung von Ki im Betrieb lässt den Stellwert nicht mehr springen
Anti-Windup	nur Begrenzung auf ±outMax/ki	Begrenzung plus bedingte Integration	Am Anschlag wird nicht weiter aufintegriert, weniger Überschwingen nach dem Aufheizen
D-Anteil	auf den Fehler	auf den Istwert	Kein D-Kick bei Sollwertsprüngen
Erster Aufruf	`return 0`	Rückgabe des aktuellen I-Anteils	Kein Einbruch des Stellwerts nach Neustart oder Umschaltung
Neu	–	`reset()`, `uebernehmen()`	Neustart nach Fehler, stoßfreie Hand→Automatik-Umschaltung

Anti-Windup im Detail

Beim Aufheizen von 20 °C auf 60 °C liegt der Stellwert minutenlang bei 100 %. Ohne Schutz summiert der I-Anteil in dieser Zeit weiter auf und muss nach dem Erreichen des Sollwerts erst wieder abgebaut werden. In der Simulation führt das zu über 20 K

Überschwingen.

Der Baustein berechnet deshalb zuerst einen Testwert `outTest = P + iNeu + D`. Liegt er über `outMax` und ist der Fehler positiv (oder unter `outMin` bei negativem Fehler), bleibt `iAnteil` unverändert. Zusätzlich ist `iAnteil` auf `outMin…outMax` begrenzt.

Stoßfreie Umschaltung

Im Handbetrieb ruft der Sketch in jedem Takt `pid.uebernehmen(handwert, istwert)` auf. Das setzt den I-Anteil auf den Handwert und `prevInput` auf den aktuellen Istwert. Beim Zurückschalten auf Automatik startet der Regler genau dort, wo der Handbetrieb aufgehört hat, statt mit einem Sprung.

Messung, SSR-Taktung und Sicherheit

Nicht-blockierende Messung

Der DS18B20 braucht bei 12 bit Auflösung 750 ms für eine Messung. Ein blockierendes `requestTemperatures()` würde den ESP32 so lange anhalten. Der Sketch nutzt deshalb `setWaitForConversion(false)` und versetzt Abholen und Starten um einen Takt:

1. Ergebnis der Messung aus dem letzten Takt mit `getTempCByIndex(0)` abholen.
2. Sofort die nächste Messung starten; sie ist beim nächsten Takt nach 1000 ms fertig.

Der Istwert ist dadurch höchstens eine Sekunde alt. Zwei Sonderwerte werden abgefangen:

- `-127 °C` (`DEVICE_DISCONNECTED_C`): Sensor fehlt oder Leitung unterbrochen → Fehlerzustand.
- `85 °C` beim allerersten Wert: Einschaltwert des Sensors, keine echte Messung → verworfen.

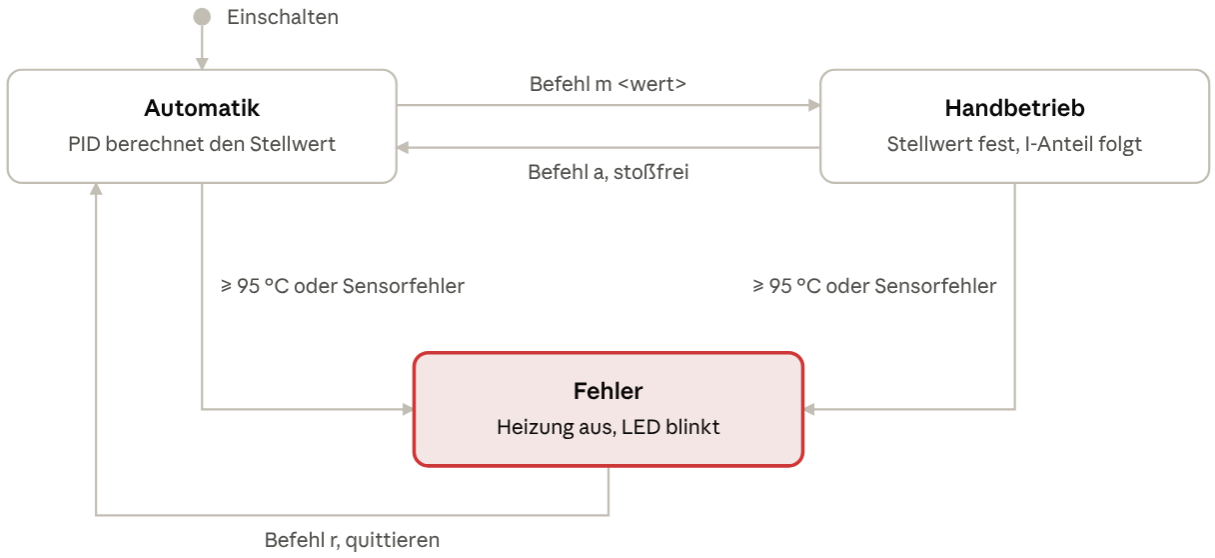
Zeit-Proportional-Ansteuerung des SSR

Ein SSR kennt nur Ein und Aus. Der Stellwert wird deshalb als Einschaltanteil eines 2-s-Fensters umgesetzt: Bei 30 % ist die Heizung 0,6 s ein und 1,4 s aus. Die Wärmeträgheit glättet das vollständig.

Einschaltzeiten unter 50 ms werden auf 0 gesetzt, Auszeiten unter 50 ms auf volle Leistung. Ein Zero-Cross-SSR an 50 Hz schaltet in Schritten einer Netzhalbwelle (10 ms); das 2-s-Fenster ergibt damit eine Auflösung von 0,5 %.

Betriebsarten

Ein Fehler sperrt die Heizung, bis er quittiert wird



Betriebsarten · 3 Zustände

Nach dem Einschalten läuft der Regler in Automatik. Aus Automatik und Handbetrieb führen Übertemperatur und Sensorfehler in den Fehlerzustand; nur `r` führt zurück, und der Regler startet dann mit leerem I-Anteil.

Sicherheitsfunktionen

Auslöser	Reaktion	Zurücksetzen
Istwert ≥ 95 °C	Heizung aus, Modus FEHLER, LED blinkt	Befehl `r`
Sensor getrennt (−127 °C)	wie oben	Befehl `r`
Kein Sensor beim Start	wie oben	Neustart mit angeschlossenem Sensor
Booten / Reset	Ausgang sofort LOW in `setup()`	–

Bedienung

Bedient wird über den Seriellen Monitor der Arduino IDE mit 115200 Baud und Zeilenende »Neue Zeile« oder »CR«. Jeder Befehl ist ein Buchstabe, gefolgt von einem optionalen Wert.

Befehl	Beispiel	Wirkung
`s <wert>`	`s 60`	Sollwert in °C (0–90)
`p <wert>`	`p 8`	Kp setzen, Einheit %/K
`i <wert>`	`i 0.05`	Ki setzen, Einheit %/(K·s)
`d <wert>`	`d 20`	Kd setzen, Einheit %·s/K
`m <wert>`	`m 30`	Handbetrieb mit festem Stellwert in %
`a`	`a`	Zurück in den Automatikbetrieb (stoßfrei)
`r`	`r`	Fehler quittieren
`w`	`w`	Sollwert und Kp/Ki/Kd im Flash speichern
`?`	`?`	Statusübersicht ausgeben

Dezimalwerte mit Punkt eingeben (`0.05`, nicht `0,05`). Ohne `w` gehen geänderte Werte beim nächsten Neustart verloren.

Serieller Plotter

Pro Takt gibt der Sketch eine Zeile im Format `Soll:60.00,Ist:59.81,Stell:23.4` aus. Der Serielle Plotter der Arduino IDE erkennt die Namen und zeichnet drei Kurven. Meldungen beginnen mit `#` und werden vom Plotter ignoriert.

Für eine Auswertung in Excel oder Python den Seriellen Monitor mitschneiden und die Zeilen ohne `#` als CSV einlesen.

Inbetriebnahme und Reglereinstellung

Die Startwerte Kp = 8, Ki = 0,05, Kd = 20 sind ein Ausgangspunkt. Jede Heizstrecke (Leistung, Wassermenge, Sensorposition) braucht eigene Werte.

Erster Test ohne Last

- [ ] Sketch hochladen, Seriellen Monitor öffnen, Statusausgabe prüfen
- [ ] Sensor in der Hand erwärmen: Istwert steigt plausibel

- [ ] `m 50` eingeben: LED leuchtet 1 s und ist 1 s aus
- [ ] Sensor abziehen: Meldung »Sensor nicht gefunden«, Heizung aus
- [ ] Sensor wieder anstecken, `r`, dann `a`

Strecke kennenlernen (Sprungantwort)

1. Im Handbetrieb mit `m 30` heizen und den Verlauf im Seriellen Plotter aufzeichnen.
2. Die **Totzeit** ablesen: Zeit vom Einschalten, bis der Istwert erkennbar steigt.
3. Die **Halteleistung** ablesen: Stellwert, bei dem die Temperatur am Sollwert stehen bleibt. Das ist später der Wert, auf den der I-Anteil einschwingt.

Lange Totzeiten (Sensor weit weg von der Heizung, viel Wasser) verlangen kleinere Kp und Ki.

Parameter einstellen

1. Mit reinem P-Regler beginnen: `i 0`, `d 0`, `p 5`.
2. Kp erhöhen, bis die Temperatur nach einem Sollwertsprung leicht um den Sollwert pendelt. Dann Kp auf etwa 60 % davon zurücknehmen. Eine bleibende Abweichung von 1–3 K ist hier normal.
3. Ki langsam erhöhen, bis die bleibende Abweichung innerhalb weniger Minuten verschwindet. Zu viel Ki erzeugt langsames Schwingen.
4. Kd erhöhen, bis das Überschwingen nach dem Aufheizen kleiner wird. Zu viel Kd macht den Stellwert unruhig, besonders bei Messrauschen.
5. Mit `w` speichern.

Beobachtung	Ursache	Abhilfe
Istwert bleibt dauerhaft unter Sollwert	Ki = 0 oder zu klein	Ki erhöhen
Großes Überschwingen nach dem Aufheizen	Kd zu klein oder Ki zu groß	Kd erhöhen, Ki senken
Langsames Schwingen mit Minuten-Periode	Ki zu groß	Ki halbieren
Schnelles Schwingen	Kp zu groß bei langer Totzeit	Kp senken
Stellwert zappelt stark	Kd zu groß bei Rauschen	Kd senken

Die Wirkung jedes Schritts lässt sich vorher im Browser-Simulator ausprobieren; dort zeigt die Copy-Funktion die passenden Befehle für den Seriellen Monitor.

Browser-Simulator

Der PID-Temperatur Simulator rechnet dieselbe `PID::update()`-Logik wie der Sketch gegen ein Modell eines Heizstabs in 1 l Wasser. Er ist für den Unterricht gedacht: Parameter verstellen, Funktionen abschalten, Wirkung sofort sehen. Die Seite ist privat, bis sie über das Teilen-Menü freigegeben wird.

Streckenmodell

Größe	Wert
Wärmekapazität Heizkörper	800 J/K
Wärmekapazität Wasser (1 l)	4200 J/K
Wärmeübergang Heizkörper → Wasser	20 W/K
Wärmeverlust an die Umgebung	4 W/K (Deckel offen: 12 W/K)
Umgebungstemperatur	20 °C
Heizleistung	800 W (einstellbar 200–2000 W)
Totzeit	3 s (einstellbar 0–30 s)
Sensor-Zeitkonstante	6 s
Sensorauflösung	0,0625 K, optional ±0,15 K Rauschen

Das Modell rechnet in 0,1-s-Schritten, der Regler taktet wie im Sketch mit 1 s, das SSR mit 2-s-Fenster. Die Zeitachse ist Simulationszeit; der Zeitraffer läuft mit 1×, 5×, 20× oder 60×.

Anzeigen

- **Temperatur**: Sollwert, gemessener Istwert, wahre Wassertemperatur und ein ±1-K-Band.
- **Stellgröße & Anteile**: Stellwert sowie P-, I- und D-Anteil getrennt, darunter ein Streifen, wann das SSR eingeschaltet ist.
- **Kennwerte**: Überschwingen seit dem letzten Sollwertsprung, Ausregelzeit in ein ±1-K-Band und bleibende Abweichung über die letzten 60 s.

Die sieben Versuche

Nr.	Versuch	Einstellung	Erwartetes Ergebnis
1	Nur P: bleibende Abweichung	Kp 10, Ki 0, Kd 0	Istwert bleibt etwa 2 K unter Sollwert; bei Kp 3 fast 6 K
2	I-Anteil beseitigt die Abweichung	Kp 10, Ki 0,05	Abweichung geht gegen 0, I-Anteil pendelt auf rund 20 % ein
3	Integral-Windup	wie 2, Anti-Windup aus	Über 20 K Überschwingen
4	D-Anteil dämpft	Kp 10, Ki 0,03, Kd 60	Überschwingen nur noch wenige Zehntel K
5	D-Kick beim Sollwertsprung	vorgeheizt auf 60 °C, D auf Fehler, SSR stufenlos	Sprung auf 80 °C lässt D kurz auf mehrere hundert % springen
6	Totzeit bringt Schwingen	Kp 60, Ki 0,5, Totzeit 15 s	Dauerhafte Schwingung um den Sollwert
7	Störgröße ausregeln	wie 4, vorgeheizt auf 60 °C	Nach »Kaltes Wasser« oder »Deckel offen« führt der I-Anteil die Leistung nach

Die Ergebnisse von Versuch 1–4 und 6 wurden vor der Veröffentlichung mit dem Modell durchgerechnet.

Fehlersuche und Erweiterungen

Symptom	Mögliche Ursache	Prüfen
»Kein DS18B20 gefunden« beim Start	Pull-up fehlt, DATA an falschem Pin	4,7 kΩ zwischen DATA und 3,3 V, `PIN_ONEWIRE`
Istwert springt auf −127 °C	Wackelkontakt, Kabel zu lang	Leitung kürzen oder Pull-up auf 2,2 kΩ verkleinern
Erster Wert 85 °C	Normaler Einschaltwert	Wird verworfen; bleibt er stehen, Spannung am Sensor prüfen
Heizung schaltet nie	SSR-Eingang verpolt, Stellwert 0	LED an GPIO 2 leuchtet? Mit `m 100` testen

Symptom	Mögliche Ursache	Prüfen
Heizung schaltet nie aus	SSR durchlegiert	Lastkreis spannungsfrei machen, SSR tauschen
Serielle Befehle ohne Wirkung	Kein Zeilenende im Monitor	Zeilenende auf »Neue Zeile« stellen
Werte nach Neustart weg	Nicht gespeichert	Nach dem Einstellen `W` senden

Erweiterungsideen

- Weboberfläche oder MQTT über WLAN, z. B. zur Anbindung an SmartHomeRw
- Sollwertrampen (z. B. 2 K/min) statt harter Sprünge
- Zweiter Sensor direkt am Heizkörper als Übertemperaturschutz
- Display (OLED oder Nextion) mit Soll- und Istwert
- Autotuning nach dem Relais-Verfahren (Åström-Hägglund)
- Datenlogging auf SD-Karte oder in den Flash